

BattleGrid

Software Design Specification

24.04.2026

Serkan EROL
150122619

Yusuf Emir ONAN
524125010

Başak AKYÜZ
150122827

Prepared for
CSE3044 Software Engineering Term Project

Table of Contents

1. Introduction	4
1.1. Purpose	4
1.2. Statement of scope	4
1.3. Software context.....	5
1.4. Major constraints.....	5
1.5. Definitions.....	6
1.6. Acronyms and Abbreviations	6
1.7. References	6
2. Design Consideration	6
2.1. Design Assumptions and Dependencies.....	7
2.2. General Constraints	7
2.3. System Environment	7
2.4. Development Methods	8
3. Architectural and component-level design	9
3.1. System Structure	9
3.1.1. Architecture diagram.....	9
3.2. BattleGrid.API Component.....	10
3.2.1. Processing Narrative.....	10
3.2.2. Component Interface Description.....	10
3.2.3. Component Processing Detail	11
3.3. BattleGrid.Application Component.....	13
3.3.1. Processing Narrative.....	13
3.3.2. Component Interface Description.....	13
3.3.3. Component Processing Detail	14
3.4. BattleGrid.Domain Component.....	16
3.4.1. Processing Narrative.....	16
3.4.2. Component Interface Description.....	17
3.4.3. Component Processing Detail	17
3.5. BattleGrid.Infrastructure Component.....	19
3.5.1. Processing Narrative.....	19
3.5.2. Component Interface Description.....	20
3.5.3. Component Processing Detail	20
3.6. BattleGrid.Contracts Component	22

3.6.1.	Processing Narrative.....	23
3.6.2.	Component Interface Description.....	23
3.6.3.	Component Processing Detail	23
3.7.	3.7 BattleGrid.Tests Component.....	24
3.7.1.	Processing Narrative.....	24
3.7.2.	Component Interface Description.....	25
3.7.3.	Component Processing Detail	25
3.8.	BattleGrid.Console Component	26
3.8.1.	Processing Narrative.....	26
3.8.2.	Component Interface Description.....	26
3.8.3.	Component Processing Detail	27
3.9.	Interaction Diagrams	27
4.	Restrictions, limitations, and constraints.....	30
5.	Conclusion.....	31
6.	Decision Log	31

1. Introduction

This section provides an overview of the entire design document. This document describes the architectural, data, and behavioral design of the BattleGrid system based on the requirements previously defined in the Requirements Specification Document (RSD).

1.1. *Purpose*

This Design Specification Document (DSD) describes the internal design and architecture of the BattleGrid system. It translates the requirements defined in the RSD into a structured technical solution that guides implementation.

It is intended for developers, project team members, and instructors to understand how the system will be built.

1.2. *Statement of scope*

Major Inputs

- User credentials (username, password)
- Matchmaking requests
- Ship placement data
- Attack coordinates

Processing Functionality

- Authentication and authorization
- Queue management
- Rank-based matchmaking
- Game session creation and management
- Ship placement validation
- Turn-based gameplay enforcement
- Attack processing and result calculation
- Ranking updates and leaderboard generation
- Match history storage

Outputs

- Authentication results
- Matchmaking status
- Game state updates (waiting for players, ship placement, in progress etc.)
- Cell state updates (hits, misses)
- Match results (win/loss)
- Leaderboard rankings
- Match history records

Functionality Ranking

Essential Requirements

- User registration and authentication
- Matchmaking system
- Game session management
- Ship placement validation
- Turn-based gameplay
- Match result calculation

Desirable Requirements

- Ranking system
- Leaderboard display
- Match history tracking
- Administrative controls (ban users, reset leaderboard)

Future Requirements

- Spectator mode
- AI opponent
- In-game chat functionality

1.3. *Software context*

BattleGrid operates as a **client–server web application**.

- The **client side** is implemented using **Blazor Server**, providing the user interface through a web browser.
- The **server side** is implemented using **ASP.NET Core (.NET 10)** and handles business logic, matchmaking, and gameplay.
- The system uses **SignalR** for real-time interaction between players and the game.
- Persistent data is stored in a **PostgreSQL (v18)** database.

The system is standalone and does not depend on external game services or engines.

1.4. *Major constraints*

The following constraints impact the design and implementation of the BattleGrid system:

- The system must be implemented as a **web-based application** accessible through modern browsers.
- The system must support **turn-based multiplayer gameplay** in real-time with strict rule enforcement.
- Development must be completed within the **time constraints of a single academic semester**.
- The system must operate under **limited concurrent user capacity**, but not less than 10, suitable for a course project.
- The system must ensure **data consistency and synchronization** between players during gameplay.

1.5. Definitions

- **SignalR:** Real-time communication library for ASP.NET
- **JSON Web Token:** An encoded and encrypted token given to authenticated clients in cookie form for authorization purposes.
- **Cookie:** A web technology to save data and state on client side in a stateless server – client connection.
- **REST API:** An API standard that follows specific architectural principles like statelessness and client-server architecture, and uses standard HTTP methods.
- **Player:** A registered user who participates in matches
- **Admin:** A privileged user who can manage players and system data
- **Matchmaking:** The process of pairing players for a game
- **Game Session:** An instance of a match between two players
- **Grid:** The board where ships are placed and attacks are made
- **Move:** A player firing a shot at a coordinate on their opponent's board.
- **Turn-Based Gameplay:** A system where players act alternately
- **Rank:** A numerical or categorical indicator of player performance used for fair matching.
- **Leaderboard:** A ranked list of players based on performance
- **Match History:** A record of past matches and their outcomes.

1.6. Acronyms and Abbreviations

- **SRS:** Software Requirements Specification
- **RSD:** Requirement Specification Document
- **DSD:** Design Specification Document
- **OS:** Operating System
- **API:** Application Programming Interface
- **UML:** Unified Modelling language
- **DB:** Database
- **HTTP:** Hypertext Transfer Protocol
- **UI:** User Interface
- **JWT:** JSON Web Token

1.7. References

- CSE3044 Software Engineering Course – SWE Project Guideline Slides
- CSE3044_DSD_Template
- IEEE Std 1016-2009 – IEEE Standard for Information Technology—Systems Design—Software Design Descriptions (SDDs)
- BattleGrid RSD report submitted by the project group
- BattleGrid Initial Report submitted by the project group

2. Design Consideration

This section describes key issues and factors considered before defining the system design.

2.1. Design Assumptions and Dependencies

The design of the BattleGrid system is based on the following assumptions and dependencies:

- **Stable Internet Connection:** Users are expected to have a stable internet connection to support real-time gameplay.
- **Client Environment:** Users will access the system through modern web browsers that support standard web technologies such as HTTP, and cookies
- **Server Availability:** The backend server is assumed to be continuously available during gameplay sessions.
- **User Knowledge:** Players are expected to understand the basic rules of the Battleship game.
- **Limited Scale:** The system is designed for a relatively small number of concurrent users, suitable for a course project.
- **Technology Dependency:**
 - The system depends on ASP.NET Core (.NET 10), SignalR, and PostgreSQL for implementation.
 - Additionally, the system depends on JWT technology and cookies to authorize authenticated users.
 - The systems depends on SwaggerUI nuget package for ASP.NET to simplify API endpoint development, testing, and documentation.

2.2. General Constraints

The following general constraints impact the system design:

- **Web-Based Operation:** The system must function entirely within a web browser environment.
- **Real-Time Communication:** Gameplay requires real-time synchronization between players via SignalR.
- **Client–Server Architecture:** All game logic and validation must be handled on the server side.
- **Data Persistence:** User data, rankings, detailed match history complete with every ship placement and move, must be stored in the database.
- **Security Requirements:** Authentication must be enforced, and passwords must be securely stored.
- **Performance Expectations:**
 - The system should respond within 200 ms to user actions in game play to maintain a smooth gameplay experience.
 - The system should respond within 2 seconds to user actions in web page such as, register, login, and logout
 - The system must not save ship placements and moves to DB in real-time to prevent performance degradation. Ship placements of a match should be saved after all ships are placed, and moves should be saved after the match ends

2.3. System Environment

The development and execution environment of the system includes:

Software Environment

- **Backend:** ASP.NET Core (.NET10)
- **Frontend:** Blazor Server

- **IDE:** Developers choice of IDE with C# support, such as Visual Studio, Rider, and VS Code
- **Database:** PostgreSQL v18
- **Database Management Software:** pgAdmin4
- **Communication:** REST APIs and SignalR
- **API UI:** SwaggerUI
- **Virtualization:** Docker is used to standardize development process between developers with different OSs.
- **Testing:** xUnit for unit testing, console app and SwaggerUI for manual testing
- **Version Control:** Git and GitHub

Hardware Environment

- **Server:** Capable of running ASP.NET Core applications
 - During development, personal computers of developers and local docker containers will be used as server
 - For deployment a cheap or free alternative will be used; such as fly.io for backend and Neon for DB deployment
- **Client devices:** Desktop computers or laptops. Mobile devices are not supported
- **Developer Systems:** Development and deployment supported on Windows, Linux, and macOS computers.

2.4. Development Methods

The development of the BattleGrid system follows an **iterative development approach**. The project is developed in small increments,

- Starting with DB implementation
- Then, basic functionality in backend:
 - DB connection, data retrieval, and saving data to DB
 - Authentication
 - Authorization
 - API endpoints
- Core Gameplay:
 - Cells (coordinates in the grid)
 - Boards, consisting of cells
 - Ship placement and placement validation logic
 - Gameplay logic
- Multiplayer:
 - Hubs created with SignalR to manage queue and game instances
- Frontend:
 - Landing, Login, Register, Dashboard pages
 - API connections to backend
 - Connections to Hubs for queueing and gameplay
- Additional features:
 - Ranking

- Leaderboard
- Admin controls added progressively
- Testing is performed continuously during development:
 - Unit tests are implemented as each component is completed
 - Manual tests are conducted via a console app for game logic and via SwaggerUI for API endpoints, throughout the development process

A formal methodology such as Agile is followed informally, focusing on incremental progress and regular validation of features. A 2 week sprint cycle is followed; 1 week for implementing new features and developing components, 1 week for testing, validating, and deciding next sprint's topic.

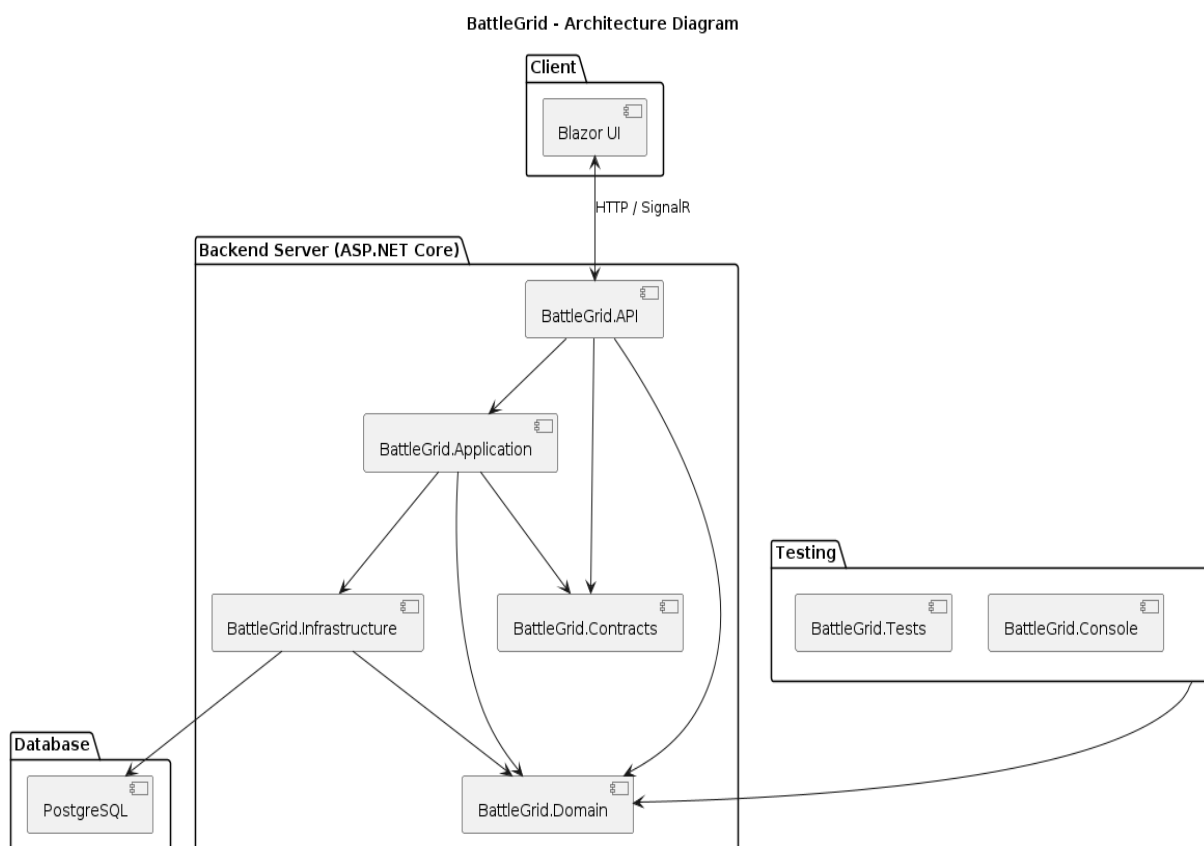
3. Architectural and component-level design

A description of the program architecture is presented.

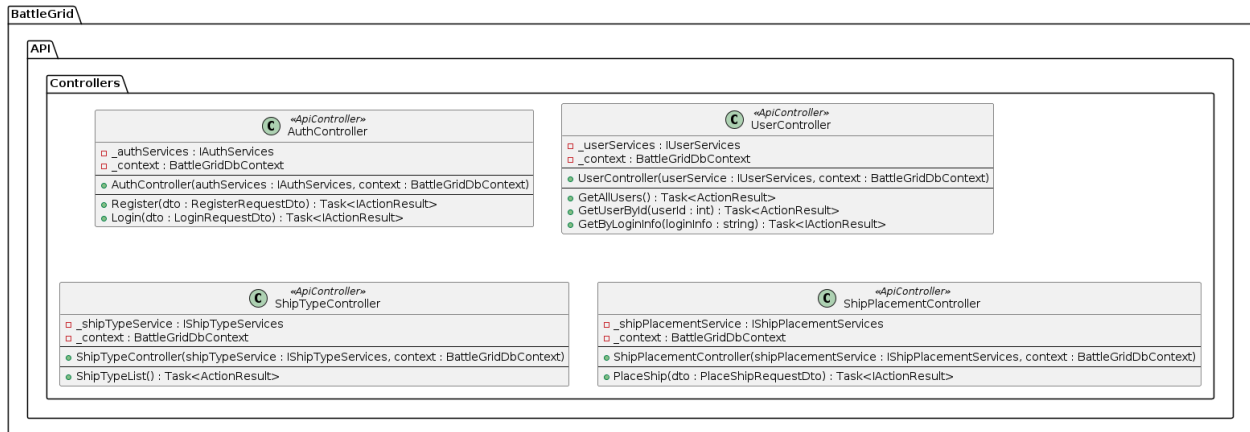
3.1. System Structure

A detailed description the system structure chosen for the application is presented.

3.1.1. Architecture diagram



3.2. BattleGrid.API Component



3.2.1. Processing Narrative

The BattleGrid.API component serves as the entry point of the system and is responsible for handling all incoming HTTP requests from clients. It exposes RESTful endpoints through controller classes and routes user actions to the appropriate application services.

This component does not contain business logic. Instead, it validates incoming requests, maps them to appropriate service calls, and returns standardized HTTP responses. It acts as a bridge between the external client and the internal application logic.

The API layer includes controllers such as AuthController, UserController, ShipTypeController, and ShipPlacementController, each responsible for a specific set of operations.

3.2.2. Component Interface Description

The API component communicates with external clients through HTTP requests and responses.

Inputs:

- HTTP requests (JSON)
- User credentials (login/register)
- Ship placement data
- User query parameters

Outputs:

- HTTP responses (JSON)
- Status codes (200, 400, 401, etc.)
- DTO-based response objects

Internally, the API interacts with the Application layer through service interfaces such as:

- IAuthServices
- IUserServices
- IShipTypeServices
- IShipPlacementServices

Additionally, the API layer has access to the database context (BattleGridDbContext) for direct data operations where needed.

3.2.3. Component Processing Detail

Each controller is responsible for a specific domain of functionality:

- **AuthController**
 - Handles user registration and login operations.
 - Accepts RegisterRequestDto and LoginRequestDto.
 - Calls authentication services and returns the result.
- **UserController**
 - Provides endpoints for retrieving user information.
 - Supports operations such as getting all users, fetching by ID, and querying by login information.
- **ShipTypeController**
 - Returns the list of available ship types for gameplay.
 - Used during the ship placement phase.
- **ShipPlacementController**
 - Handles ship placement requests from players.
 - Accepts PlaceShipRequestDto and forwards it to the application layer.

The general request flow in this component is:

1. Receive HTTP request
2. Validate input data
3. Call corresponding service via interface
4. Process result returned from service
5. Return HTTP response to client

3.2.3.1. Design Class Structure

The API component consists of controller classes that depend on service interfaces and data transfer objects (DTOs). Controllers are implemented using ASP.NET Core's ApiController structure and follow dependency injection principles.

Each controller depends on:

- A corresponding service interface from the Application layer
- The database context (BattleGridDbContext)
- Request and response DTOs from the Contracts layer

3.2.3.2. Restrictions / Limitations

The API component must not contain business logic or game rules. All domain-related logic must be handled by the Domain layer through the Application layer.

Direct database access from controllers should be minimized and preferably handled through services to maintain clean architecture principles.

3.2.3.3. Performance Considerations

The API layer is expected to handle lightweight operations such as request validation, routing, and response generation. Performance is mainly influenced by the efficiency of service calls and database operations.

As defined in the SRS, user actions such as login and matchmaking should be processed within a short time frame (approximately 2 seconds), and in-game actions should respond quickly to ensure a smooth user experience.

3.2.3.4. *Design Constraints*

The API component must follow RESTful design principles and support stateless communication. It must also ensure secure handling of authentication data and prevent unauthorized access to protected endpoints.

All endpoints must be compatible with future real-time extensions such as SignalR-based communication

3.2.3.5. *Processing Detail for Main Operations*

Register(dto)

1. Receive registration request
2. Validate input fields
3. Call authentication service
4. Store user data
5. Return success or error response

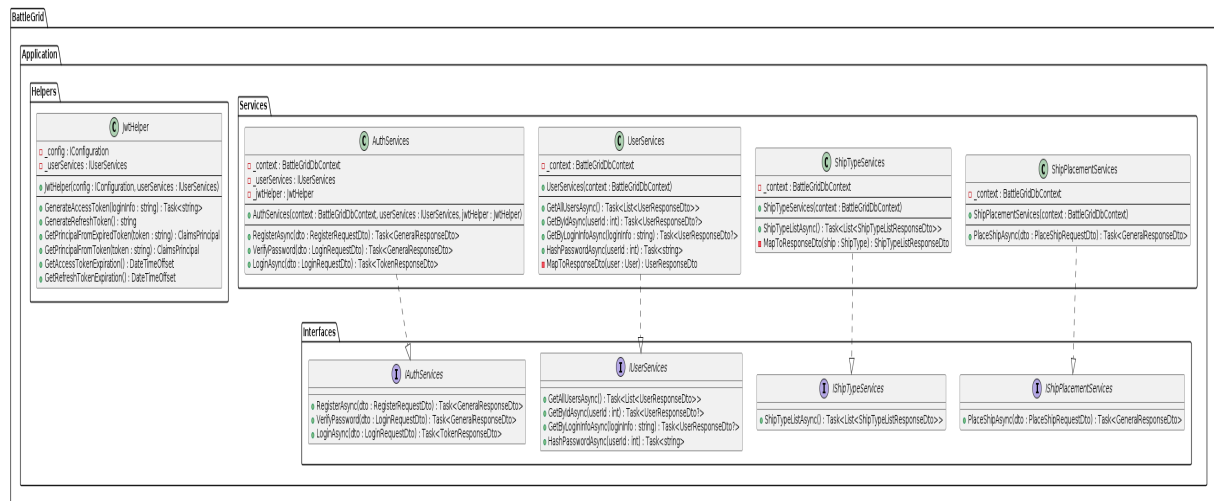
Login(dto)

1. Receive login request
2. Validate credentials
3. Call authentication service
4. Generate tokens
5. Return authentication result

PlaceShip(dto)

1. Receive placement data
2. Validate input
3. Call ship placement service
4. Update game state
5. Return result

3.3. BattleGrid.Application Component



3.3.1. Processing Narrative

The BattleGrid.Application component is responsible for coordinating the application's use cases and managing the interaction between the API layer and the Domain and Infrastructure components. It acts as an orchestration layer that contains service classes implementing the core workflows of the system.

This component does not contain low-level game logic or direct database schema definitions. Instead, it coordinates operations such as user authentication, user retrieval, ship type listing, and ship placement by invoking the appropriate domain logic and persistence mechanisms.

The Application layer is structured into three main parts:

- **Interfaces:** Define service contracts
- **Services:** Implement business workflows
- **Helpers:** Provide reusable utility logic such as JWT token handling

3.3.2. Component Interface Description

The Application component communicates with the API layer through service interfaces and with the Infrastructure layer through the database context.

Inputs:

- DTO objects from API layer (`RegisterRequestDto`, `LoginRequestDto`, `PlaceShipRequestDto`)
- User identifiers and query parameters

Outputs:

- DTO response objects (`UserResponseDto`, `TokenResponseDto`, `GeneralResponseDto`)
- Processed results of operations (authentication results, user data, ship data)

The main interfaces exposed by this component are:

- `IAuthServices`
- `IUserServices`
- `IShipTypeServices`
- `IShipPlacementServices`

Each interface defines a contract that is implemented by a corresponding service class.

3.3.3. Component Processing Detail

The Application component organizes logic through service classes:

- **AuthServices**
 - Handles user registration, password verification, and login processes
 - Uses JwtHelper to generate access and refresh tokens
 - Coordinates authentication flow between user data and token creation
- **UserServices**
 - Retrieves user-related data
 - Supports fetching all users, fetching by ID, and querying by login information
 - Handles password hashing functionality
 - Maps domain entities to response DTOs
- **ShipTypeServices**
 - Retrieves available ship types from the database
 - Maps domain objects to response DTOs
- **ShipPlacementServices**
 - Processes ship placement requests
 - Coordinates with the Domain logic to validate placements
 - Stores placement data in the database
- **JwtHelper**
 - Responsible for generating and validating JWT tokens
 - Handles token expiration and extracting user information from tokens

The general flow within this component is:

1. Receive request from API layer
2. Validate input data
3. Execute business workflow through service methods
4. Call domain logic or persistence layer if needed
5. Map results to DTOs
6. Return result to API layer

3.3.3.1. Design Class Structure

The Application component follows a service-based architecture with interface-based abstraction.

- Interfaces define contracts for each use case
- Services implement these interfaces
- Dependency injection is used to inject required dependencies such as:
 - BattleGridDbContext
 - JwtHelper
 - Other service interfaces

Each service class is responsible for a specific domain of functionality, ensuring separation of concerns.

3.3.3.2. Restrictions / Limitations

The Application component should not contain detailed game rules or low-level logic. Those responsibilities belong to the Domain layer.

It should also avoid direct HTTP handling or response formatting, which must remain in the API layer.

3.3.3.3. *Performance Considerations*

The Application layer performs moderate processing, including DTO mapping, service coordination, and token generation. Performance depends mainly on database access and domain operations.

Since most operations involve simple queries or lightweight processing, this layer is expected to perform efficiently within the project constraints.

3.3.3.4. *Design Constraints*

The Application component must follow clean architecture principles and remain independent from the presentation layer. It must rely on abstractions (interfaces) instead of concrete implementations to support flexibility and testability.

It should also be compatible with future extensions such as real-time communication through SignalR.

3.3.3.5. *Processing Detail for Main Operations*

RegisterAsync(dto)

1. Validate input
2. Check if user already exists
3. Hash password
4. Store user in database
5. Return response

LoginAsync(dto)

1. Validate input
2. Verify password
3. Generate access token
4. Generate refresh token
5. Return token response

GetAllUsersAsync()

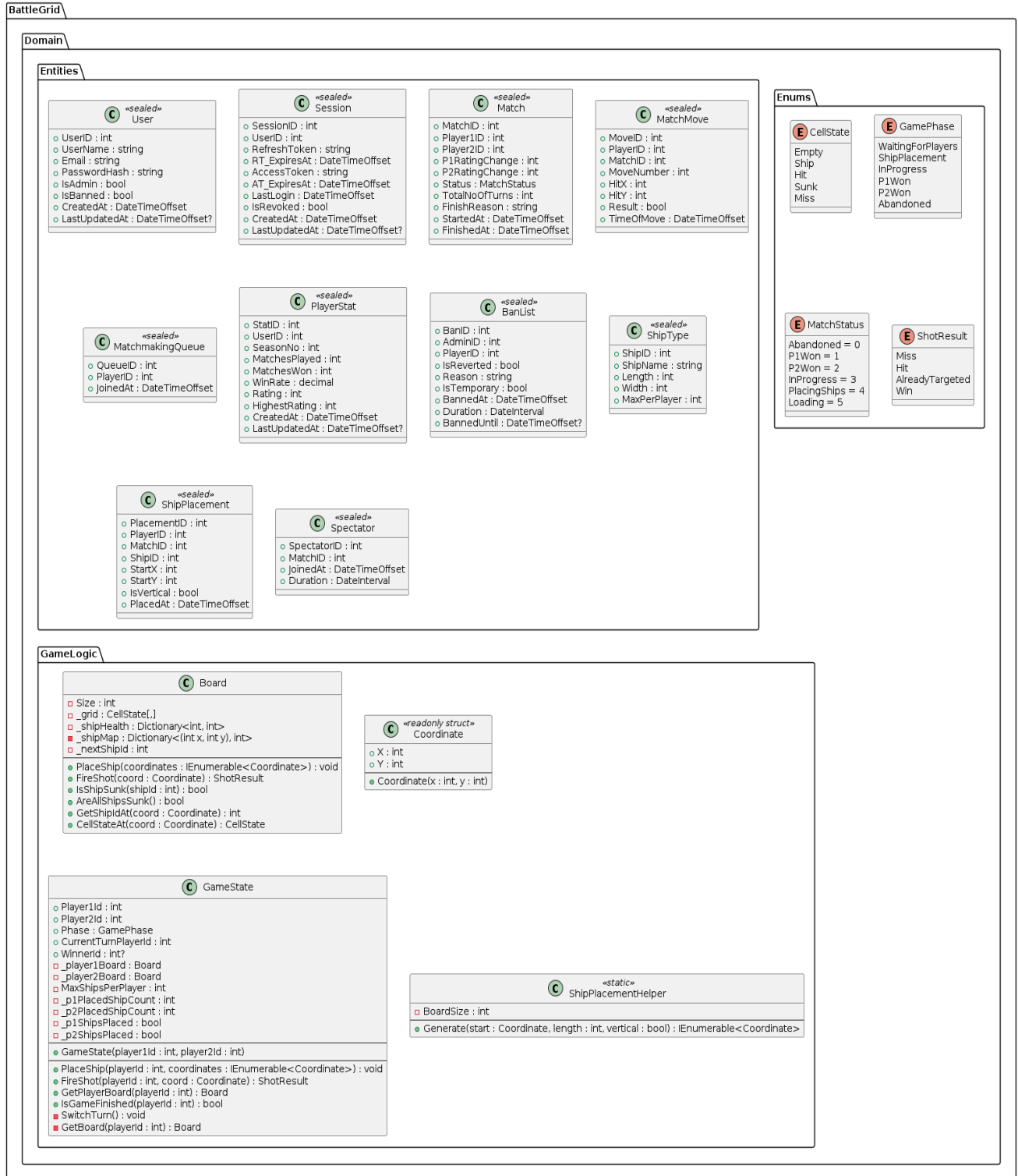
1. Query database
2. Map users to DTO
3. Return list

PlaceShipAsync(dto)

1. Validate placement request
2. Call domain logic if necessary
3. Store placement in database
4. Return result

3.4.

BattleGrid.Domain Component



3.4.1. Processing Narrative

The BattleGrid.Domain component contains the core business logic and domain model of the BattleGrid system. This component is independent from the API and database layers. Its main responsibility is to represent the main concepts of the system such as users, matches, sessions, rankings, ship placements, and game state.

The most important part of this component is the GameLogic package. It manages the rules of the Battleship game, including board representation, ship placement validation, turn management, attack processing, hit/miss detection, and game completion control. The domain layer ensures that game rules are applied consistently regardless of which external interface calls the system.

This design keeps the core game rules separate from controllers, database access, and user interface concerns. Therefore, the game logic can be tested independently using unit tests. The README also states that current unit tests focus on game logic such as coordinate validation, ship placement validation, player turn management, hit registration, and game end detection.

3.4.2. Component Interface Description

The Domain component is used mainly by the Application layer. It receives structured data such as player identifiers, coordinates, ship placement information, and attack commands. It returns domain-level results such as updated game state, shot result, current turn information, winner information, and validation outcomes.

The main input/output elements are:

Inputs:

- Player identifiers
- Match identifiers
- Ship placement coordinates
- Attack coordinates
- Current game state information

Outputs:

- Updated board state
- Shot result: Miss, Hit, AlreadyTargeted, Win
- Updated game phase
- Current turn player
- Winner information
- Match status information

The component does not directly communicate with the client, REST API, or PostgreSQL database. Instead, those responsibilities are handled by API, Application, and Infrastructure components.

3.4.3. Component Processing Detail

The GameState class coordinates the gameplay flow between two players. It stores the player IDs, current game phase, current turn, winner information, and the two player boards. During the ship placement phase, GameState.PlaceShip() places ships on the correct player board. When both players complete ship placement, the game can continue to the active gameplay phase.

The Board class represents a player's grid. It stores cell states, ship health information, and the mapping between board coordinates and ship identifiers. It validates whether a ship can be placed, records attacks, updates cell states, and checks whether all ships are sunk.

The Coordinate struct represents a position on the board using X and Y values. It is used by both ship placement and attack operations.

The ShipPlacementHelper class generates ship coordinates based on a starting coordinate, ship length, and orientation. This prevents coordinate generation logic from being duplicated in different parts of the system.

The entity classes such as User, Match, PlayerStat, ShipPlacement, MatchMove, Session, and BanList represent persistent domain data. These classes correspond to the main data concepts defined in the SRS, such as user accounts, rankings, match history, game session data, and administrative actions.

3.4.3.1. *Design Class Hierarchy for BattleGrid.Domain*

The Domain component is divided into three main packages:

BattleGrid.Domain.Entities

BattleGrid.Domain.Enums

BattleGrid.Domain.GameLogic

Entities contains persistent domain objects such as User, Match, PlayerStat, ShipPlacement, and MatchMove.

Enums contains controlled state values such as GamePhase, MatchStatus, CellState, and ShotResult.

GameLogic contains the core gameplay classes such as GameState, Board, Coordinate, and ShipPlacementHelper.

3.4.3.2. *Restrictions / Limitations*

The Domain component should not contain API-specific, UI-specific, or database-specific logic. It should not directly process HTTP requests, return HTTP responses, or execute SQL queries.

The current game logic is designed around two-player Battleship gameplay. Additional modes such as tournaments, spectators, or AI opponents may require extending this component in the future.

3.4.3.3. *Performance Issues*

The Domain component performs lightweight in-memory operations such as coordinate validation, grid updates, and turn checks. These operations are expected to be fast because the board size and number of ships are limited.

The performance of this component is mainly affected by the number of active game sessions and how often game state operations are executed. Since the SRS targets at least 10 concurrent active players, the current in-memory game logic is suitable for the expected project scale.

3.4.3.4. *Design Constraints*

The Domain component must enforce the official game rules consistently. Invalid actions such as attacking outside the board, attacking the same cell repeatedly, placing overlapping ships, or playing out of turn must be prevented.

The component must also remain independent from infrastructure details so that the same logic can be used by REST endpoints, future SignalR hubs, console testing, and unit tests.

3.4.3.5. *Processing Detail for Main Operations*

PlaceShip(playerId, coordinates)

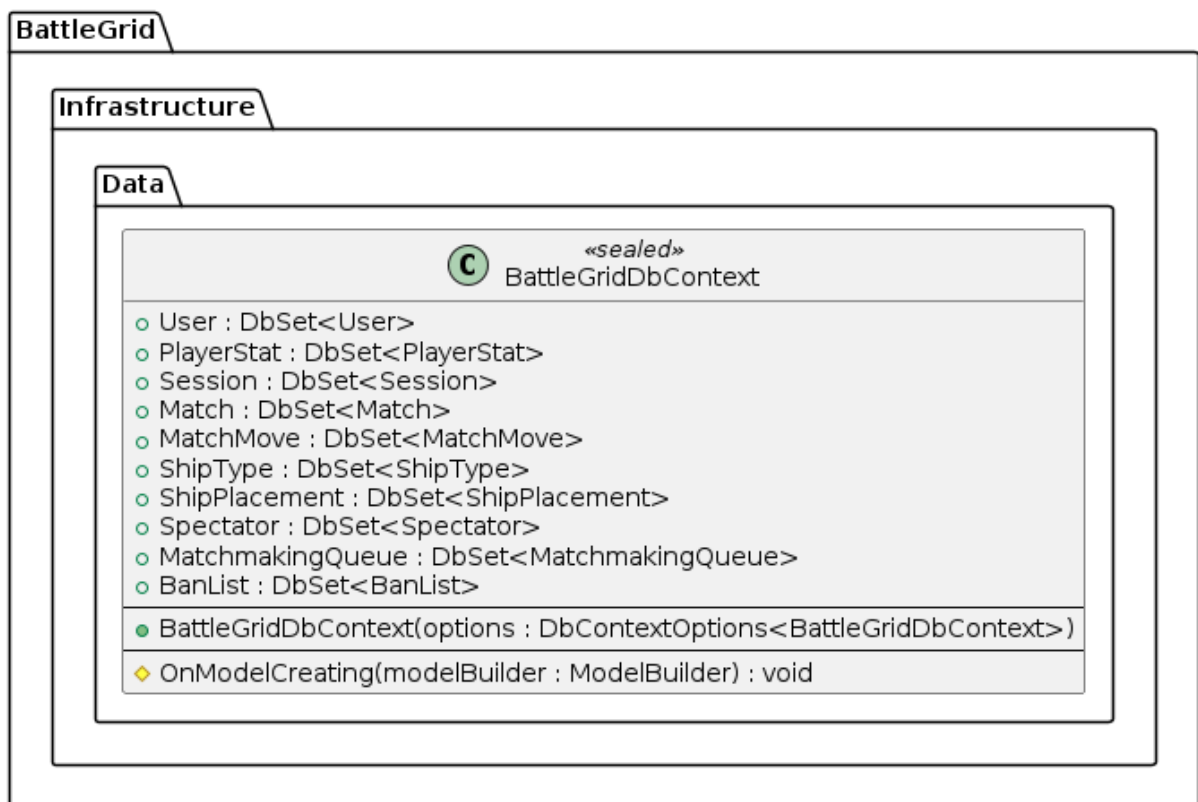
1. Determine the board that belongs to the given player.
2. Validate the given coordinates.
3. Check whether the ship is inside the board.
4. Check whether the ship overlaps with an existing ship.
5. Place the ship on the board.

6. Update the player's placed ship count.
7. If both players completed placement, allow the game to continue.

FireShot(playerId, coordinate)

1. Check whether the game is in progress.
2. Check whether the requesting player has the current turn.
3. Select the opponent's board.
4. Validate the target coordinate.
5. Apply the shot to the opponent board.
6. Return Miss, Hit, AlreadyTargeted, or Win.
7. If the result is Win, update the winner and game phase.
8. If the result is Miss, switch the turn to the other player.

3.5. BattleGrid.Infrastructure Component



3.5.1. Processing Narrative

The BattleGrid.Infrastructure component is responsible for data persistence and database interaction. It provides the connection between the application and the PostgreSQL database and handles all operations related to storing and retrieving data.

This component uses Entity Framework Core as the Object-Relational Mapping (ORM) tool. It maps domain entities to database tables and manages database transactions. The central class in this component is BattleGridDbContext, which represents the database session and provides access to all tables.

The Infrastructure layer ensures that data such as users, matches, player statistics, ship placements, and game actions are stored and retrieved in a consistent and structured manner.

3.5.2. Component Interface Description

The Infrastructure component is used internally by the Application layer and does not directly interact with external clients.

Inputs:

- Entity objects (User, Match, PlayerStat, ShipPlacement, etc.)
- Query requests from Application services

Outputs:

- Database records
- Retrieved entity objects
- Persisted changes

The main interaction is performed through the BattleGridDbContext class, which exposes DbSet properties corresponding to each domain entity.

3.5.3. Component Processing Detail

The BattleGridDbContext class defines the structure of the database and provides access to all data entities. Each DbSet represents a table in the database:

- User
- PlayerStat
- Session
- Match
- MatchMove
- ShipType
- ShipPlacement
- Spectator
- MatchmakingQueue
- BanList

These sets correspond directly to the domain entities defined in the Domain layer.

The OnModelCreating method is used to configure entity relationships, constraints, and database mappings using the Entity Framework Core fluent API.

The general data flow in this component is:

1. Receive request from Application layer

2. Translate request into database query
3. Execute query via Entity Framework
4. Retrieve or update data
5. Return results to Application layer

3.5.3.1. Design Class Structure

The Infrastructure component is centered around a single database context class:

- BattleGridDbContext

This class:

- Extends the Entity Framework Core DbContext
- Defines all entity sets (DbSet<T>)
- Manages database configuration and relationships

The design follows a centralized data access pattern, where all database operations are handled through the context object.

3.5.3.2. Restrictions / Limitations

The Infrastructure component must not contain business logic or game rules. Its sole responsibility is data access and persistence.

All validation and rule enforcement must be handled in the Domain or Application layers.

3.5.3.3. Performance Considerations

Performance in this component depends on database operations such as queries, inserts, and updates. Efficient indexing and proper query design are important to ensure acceptable performance.

As the system grows, large tables such as match history and player statistics may require optimization techniques such as indexing and query tuning.

3.5.3.4. Design Constraints

The Infrastructure component must remain compatible with PostgreSQL and Entity Framework Core.

It must ensure data integrity, support relational mapping between entities, and maintain consistency across all database operations.

3.5.3.5. Processing Detail for Main Operations

Create Entity

1. Receive entity from Application layer
2. Add entity to corresponding DbSet
3. Save changes to database

Read Entity

1. Receive query request
2. Execute LINQ query
3. Return matching records

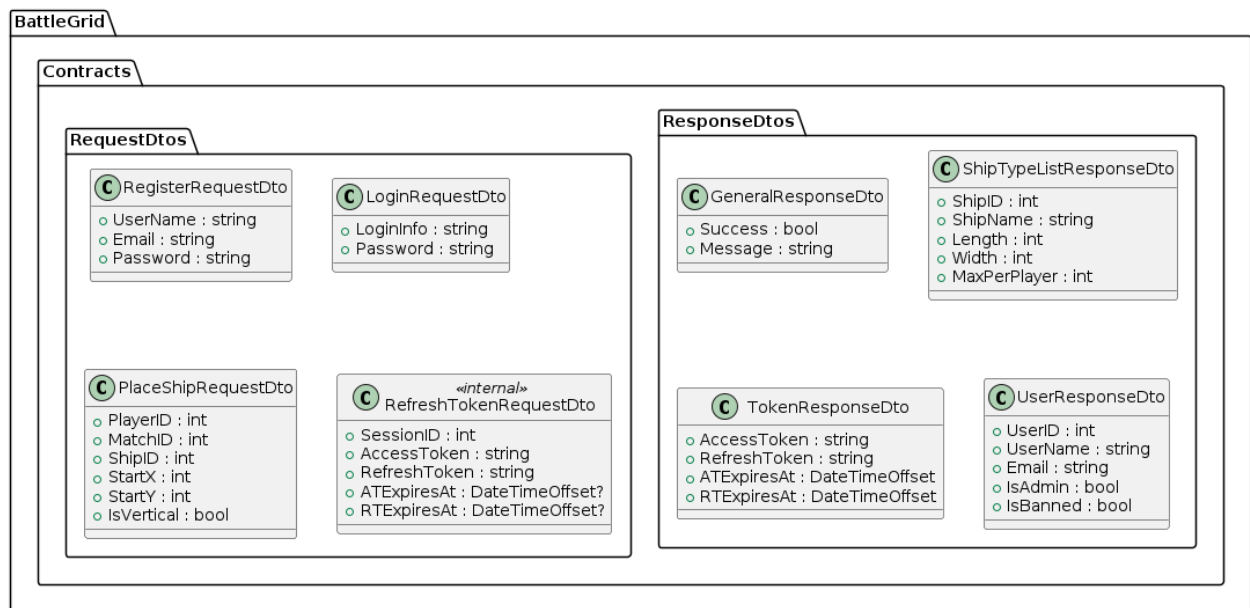
Update Entity

1. Retrieve entity from database
2. Modify fields
3. Save changes

Delete Entity

1. Retrieve entity
2. Remove from DbSet
3. Save changes

3.6. *BattleGrid.Contracts Component*



3.6.1. Processing Narrative

The BattleGrid.Contracts component defines the data transfer objects (DTOs) used for communication between different layers of the system, especially between the API and Application components.

This component ensures that data exchanged between layers is structured, consistent, and independent from the internal domain models. By using DTOs, the system avoids exposing domain entities directly to external clients.

The Contracts layer is divided into two main groups:

- **Request DTOs:** Used for incoming data from clients
- **Response DTOs:** Used for outgoing data returned to clients

3.6.2. Component Interface Description

The Contracts component acts as a shared data structure layer.

Inputs:

- Data from external clients (via API)
- Data from Application services

Outputs:

- Structured request objects for Application layer
- Structured response objects for API layer

Main DTO categories include:

- Authentication DTOs (RegisterRequestDto, LoginRequestDto)
- Game-related DTOs (PlaceShipRequestDto)
- Token-related DTOs (TokenResponseDto, RefreshTokenRequestDto)
- General response DTOs (GeneralResponseDto, UserResponseDto, ShipTypeListResponseDto)

3.6.3. Component Processing Detail

The Contracts component does not contain business logic or processing behavior. It only defines data structures.

- **Request DTOs**
 - Represent data sent by the client
 - Used by controllers to pass input to Application services
- **Response DTOs**
 - Represent structured responses returned to the client
 - Created by Application services and returned through the API layer

For example:

- RegisterRequestDto contains user registration data such as username, email, and password
- PlaceShipRequestDto contains ship placement information such as coordinates and orientation
- TokenResponseDto contains authentication tokens and expiration data
- UserResponseDto contains user profile information

3.6.3.1. Design Class Structure

The Contracts component is organized into two main packages:

- BattleGrid.Contracts.RequestDtos
- BattleGrid.Contracts.ResponseDtos

Each package contains simple data classes with public properties and no behavior.

3.6.3.2. *Restrictions / Limitations*

The Contracts component must not contain any business logic, database logic, or validation rules. Its sole responsibility is to define data structures.

All validations and transformations must be handled in the Application or Domain layers.

3.6.3.3. *Performance Considerations*

DTOs are lightweight structures and do not introduce significant performance overhead. However, excessive or unnecessary data fields should be avoided to minimize payload size in API responses.

3.6.3.4. *Design Constraints*

The Contracts component must remain independent from Domain entities to preserve loose coupling between layers.

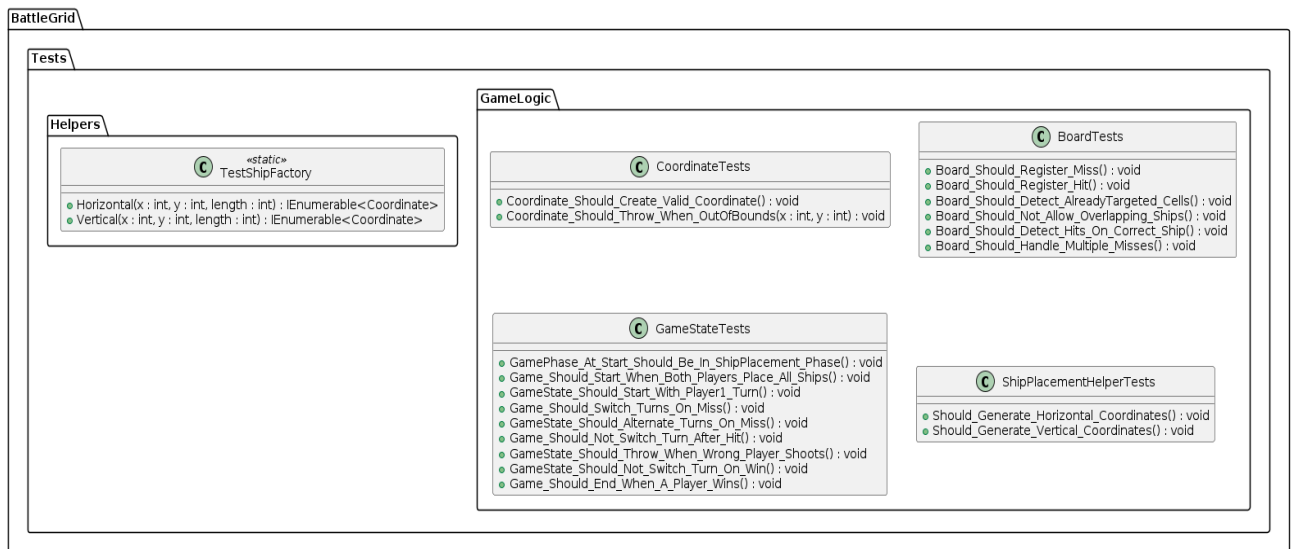
It must also be stable, as changes in DTO structures may affect multiple components such as API controllers and frontend clients.

3.6.3.5. *Processing Detail for Data Flow*

Client → API → Request DTO → Application

Application → Response DTO → API → Client

3.7. *3.7 BattleGrid.Tests Component*



3.7.1. *Processing Narrative*

The BattleGrid.Tests component is responsible for validating the correctness of the system's core logic through automated unit tests. It focuses primarily on testing the Domain layer, especially the game logic implemented in classes such as GameState, Board, Coordinate, and ShipPlacementHelper.

This component ensures that the core gameplay mechanics behave as expected under different scenarios, including valid and invalid inputs. By isolating the logic from the API and database layers, the system can be tested independently and reliably.

3.7.2. Component Interface Description

The Tests component does not interact with external clients or other system layers during runtime. Instead, it interacts directly with Domain classes in a controlled test environment.

Inputs:

- Predefined test data
- Domain objects (GameState, Board, Coordinate, etc.)
- Helper-generated coordinates

Outputs:

- Test results (pass/fail)
- Assertions validating expected behavior

3.7.3. Component Processing Detail

The Tests component is organized into two main parts:

- **Helpers**
- **GameLogic Tests**

TestShipFactory

- Generates predefined ship coordinate sets
- Provides reusable test data for ship placement scenarios

CoordinateTests

- Validates correct coordinate creation
- Ensures invalid coordinates throw exceptions

BoardTests

- Tests board behavior such as:
 - Hit and miss registration
 - Preventing overlapping ships
 - Detecting already targeted cells
 - Handling multiple shots correctly

GameStateTests

- Tests overall game flow logic:
 - Initial game phase
 - Turn management
 - Switching turns after miss
 - Preventing invalid player actions
 - Detecting win conditions

ShipPlacementHelperTests

- Verifies coordinate generation logic for ships
- Tests both horizontal and vertical placements

3.7.3.1. *Design Class Structure*

The Tests component is divided into:

BattleGrid.Tests.Helpers

BattleGrid.Tests.GameLogic

Each test class corresponds to a specific Domain class and validates its behavior using unit testing methods.

3.7.3.2. *Restrictions / Limitations*

The Tests component is designed for development and validation purposes only. It is not part of the production system.

It does not test API endpoints or database operations in its current form. The focus is limited to Domain logic testing.

3.7.3.3. *Performance Considerations*

The performance of the Tests component is not critical for production usage. However, efficient test execution is important during development to provide quick feedback.

3.7.3.4. *Design Constraints*

The Tests component must remain independent from external systems such as databases and network communication. Tests should be deterministic and reproducible.

Mocking or isolation techniques should be used if dependencies are introduced in the future.

3.7.3.5. *Processing Detail for Test Execution*

1. Initialize test data
2. Create domain object instance
3. Execute method under test
4. Compare result with expected outcome
5. Assert correctness
6. Report test result

3.8. *BattleGrid.Console Component*

3.8.1. *Processing Narrative*

The BattleGrid.Console component is a supporting tool used for manually testing and verifying the game logic during development. It allows developers to simulate a game session without using the API, database, or external interfaces.

This component directly interacts with the Domain layer and executes core gameplay operations such as ship placement, turn handling, and attack processing.

3.8.2. *Component Interface Description*

Inputs:

- Console input (attack coordinates entered by the user)

Outputs:

- Printed game state
- Hit/miss results
- Turn information
- Game end result

3.8.3. Component Processing Detail

The console application initializes a game scenario with predefined ship placements. Players take turns entering coordinates, and the application forwards these inputs to the Domain game logic.

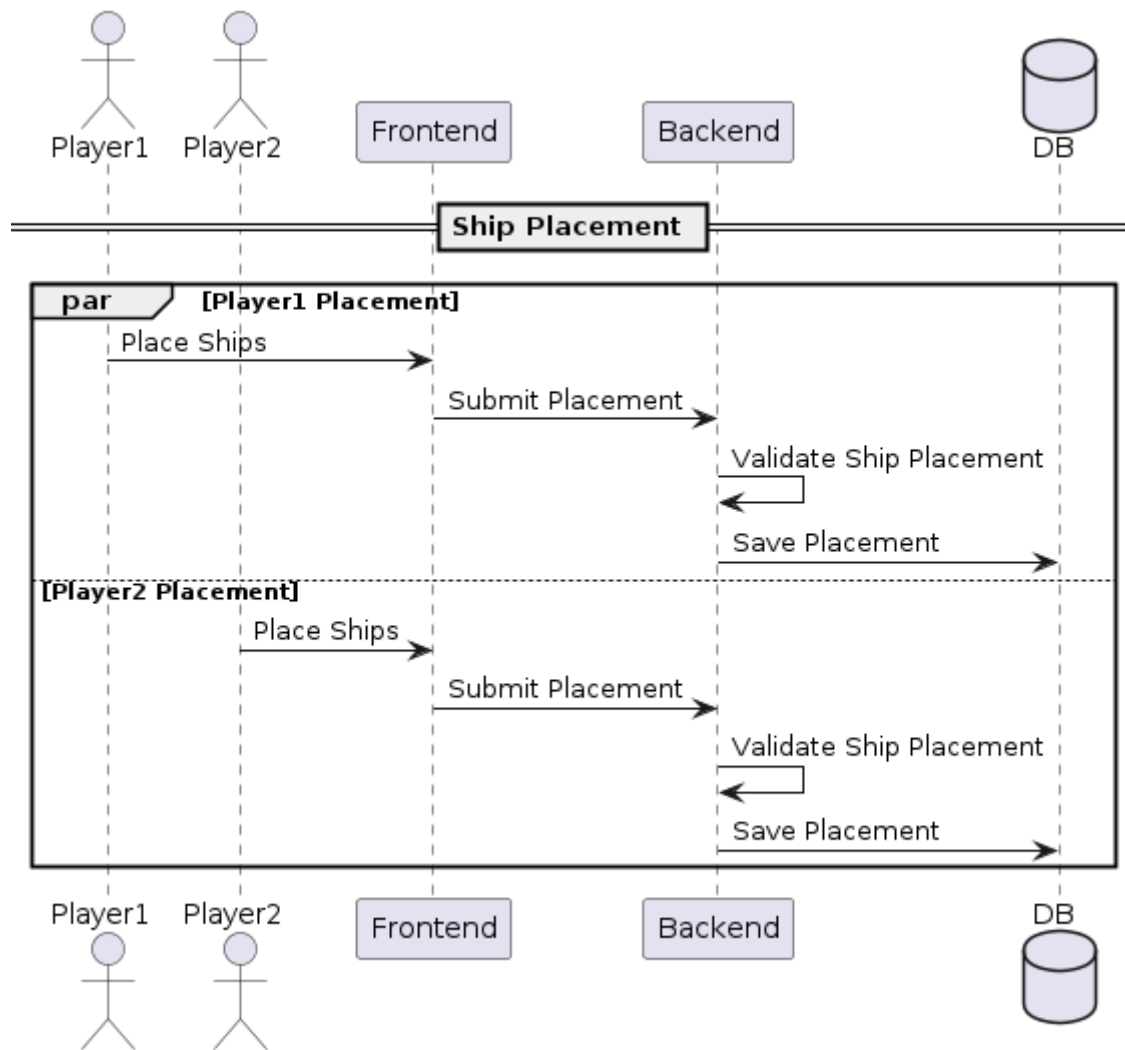
1. Initialize game state
2. Place ships for both players
3. Start gameplay loop
4. Read input from console
5. Call Domain methods (FireShot, etc.)
6. Print results
7. Continue until game ends

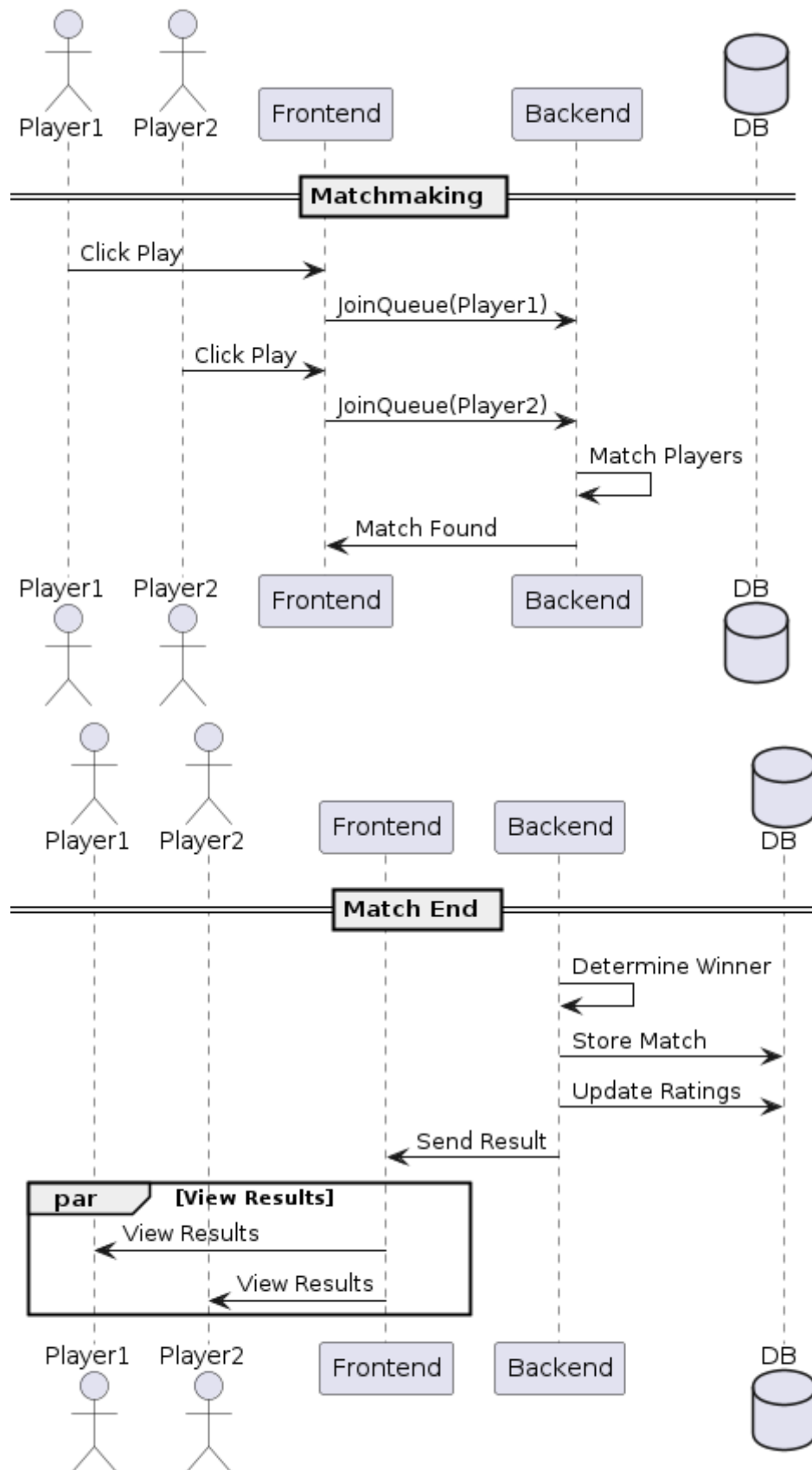
3.8.3.1. Restrictions / Limitations

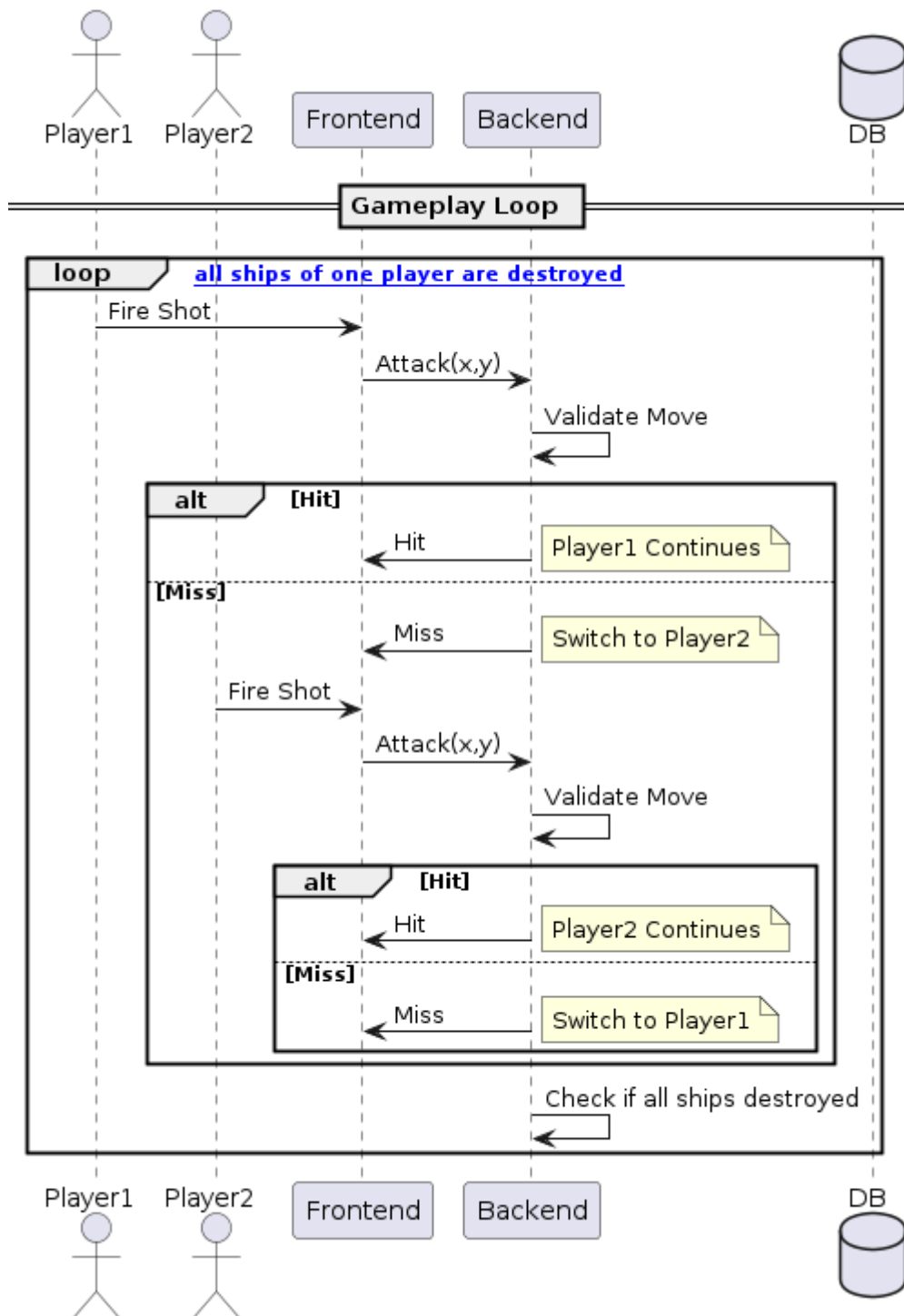
The Console component is not part of the production system. It does not include authentication, matchmaking, persistence, or network communication. Its purpose is limited to development and debugging.

3.9. Interaction Diagrams

A sequence diagram, for each use case the component realizes, is presented.







4. Restrictions, limitations, and constraints

Special restrictions, limitations, and constraints which impact the design and implementation of the software are noted here.

Limited Scalability: The system is designed for a small number of concurrent users due to the scope of a course project. This affects architectural decisions by avoiding distributed systems or load balancing mechanisms.

In-Memory Game State Management: Active game sessions are stored in server memory instead of being persisted after each move. This design choice improves performance and reduces database load, but introduces a limitation where ongoing matches may be lost in case of connection or server failure.

Development Time Constraint: The system must be completed within a single academic semester. This limits the implementation of advanced features and leads to prioritization of core gameplay functionality over additional enhancements.

Security Scope Limitation: The system implements basic security measures such as authentication and password hashing. Advanced security features (e.g., multi-factor authentication, intrusion detection) are not included due to project scope constraints.

5. Conclusion

The design outlined in this document provides the technical framework for BattleGrid, translating the requirements into a functional client-server web application. The implementation roadmap prioritizes essential requirements to ensure a viable product is delivered within the semester's academic constraints. This document concludes the design phase and serves as the definitive guide for the project team during the upcoming development and testing iterations.

6. Decision Log

Backend Technology Selection

- Decision Made: Use ASP.NET Core (.NET 10) as the backend framework.
- Alternatives Considered: Node.js with Express, Spring Boot (Java), Django (Python)
- Rationale: The team has prior experience with C# and .NET. ASP.NET Core integrates natively with SignalR, Entity Framework Core, and has strong typing which benefits a game with complex state management.
- Trade-offs: Heavier framework than Node.js; requires more setup. However, the built-in dependency injection and middleware pipeline justify the overhead.
- Risks or Technical debt: Locked into the .NET ecosystem. Migration to another backend would be costly.

Real-Time Communication

- Decision Made: Use SignalR for real-time player communication during gameplay.
- Alternatives considered: Raw WebSockets, Long polling / HTTP polling, Server-Sent Events (SSE)

- Rationale: SignalR abstracts transport negotiation (WebSocket, SSE, long polling fallback) and integrates directly with ASP.NET Core. It simplifies hub-based game session management significantly.
- Trade-offs: Adds a dependency and slightly more overhead than raw WebSockets. Less control over the transport layer.
- Risks or Technical debt: SignalR hubs managing game state in memory means server restarts wipe active sessions. Persistent game state storage would be needed for production resilience.

Frontend Framework

- Decision Made: Use Blazor Server as the frontend framework.
- Alternatives Considered: React, Angular, Vue.js
- Rationale: Blazor Server allows the team to write both frontend and backend in C#, reducing context switching and leveraging existing team skills. It also shares models with the backend directly.
- Trade-offs: Blazor Server is less mature than React/Angular ecosystems. Component library support is narrower. Mobile is explicitly not supported, which limits audience.
- Risks or Technical debt: Blazor Server maintains a persistent SignalR connection per client.

Database

- Decision Made: Use PostgreSQL v18 as the relational database.
- Alternatives Considered: MS SQL Server, MySQL
- Rationale: PostgreSQL is open-source, production-grade, and cross-platform. It supports complex queries needed for leaderboards and match history
- Trade-offs: Requires pgAdmin4 for management. Slightly steeper learning curve than MySQL for some team members.
- Risks or Technical debt: v18 is a recent version; some tooling may not have caught up.